

Setup — Supabase + Vercel

This is the migration from the original localStorage-only prototype to a real backend: Postgres (via Supabase) for events/tasks, Supabase Auth for real per-user login, and a Vercel Python function standing in for `server.py`'s local `/api/chat` proxy. Everything below is a one-time setup you do once you have a Supabase account and a Vercel account — I can't create those accounts for you, but every code change needed to talk to them is already done.

Budget about 15 minutes for all of this the first time through.

What changed, in one paragraph

Every page still loads React/ReactDOM/Babel/Supabase from a CDN `<script>` tag and compiles JSX in the browser — there's still no build step, no `npm install`, no bundler. The only new file every page loads is `supabase-client.js`, which is the one place that talks to your Supabase project; every page calls its `Cal.*` functions (`Cal.loadEvents()`, `Cal.saveManualEvent()`, `Cal.setTaskChecked()`, etc.) instead of reading and writing `localStorage` directly. `login.html` is new. Chat history, the voice-reply mute toggle, and light/dark theme stay in `localStorage` per-device — see the "Not migrated" note at the bottom of `sql/schema.sql` for why.

1. Create the Supabase project

1. Go to supabase.com, sign in, and click **New project**. Pick any name/region/database password (save the password somewhere — you won't need it for this app, but Supabase requires one).
2. Wait for the project to finish provisioning (a minute or two).
3. Open **SQL Editor** in the left sidebar → **New query**, paste in the entire contents of `sql/schema.sql` from this folder, and click **Run**. This creates the events and tasks tables with Row Level Security policies already attached — every query the browser makes is automatically scoped to whichever user is signed in, so there's no way for one account to see another's data.
4. Open **Project Settings** → **API**. You'll need two values from this page in the next step: **Project URL** and the **anon / public** key (not the `service_role` key — that one is secret and this app never uses it).

Email/password auth is on by default for a new Supabase project, so there's nothing to configure there. If you want to skip Supabase's "confirm your email" step while testing (so signing up logs you straight in instead of waiting on a confirmation email), go to **Authentication** → **Providers** → **Email** and turn off **Confirm email** — fine for personal use, worth turning back on before sharing the app with anyone else.

2. Point the app at your project

Open `supabase-client.js` and replace the two placeholder values near the top:

```
const SUPABASE_URL = 'YOUR_SUPABASE_URL';
```

```
const SUPABASE_ANON_KEY = 'YOUR_SUPABASE_ANON_KEY';
```

with the **Project URL** and **anon / public** key from step 1.4. This is the only code change needed to connect the whole app to your database — all six HTML pages load this one file.

The anon key is meant to be public/client-side (that's why Supabase calls it "public") — Row Level Security in `sql/schema.sql` is what actually keeps data private, not secrecy of this key.

3. Deploy to Vercel

1. Push this folder to a GitHub repo (or connect Vercel directly to wherever it already lives).
2. In the [Vercel dashboard](#), **Add New → Project**, import that repo. No framework preset needed — Vercel serves the `.html` files as static files automatically, and auto-detects `api/chat.py` as a Python serverless function because it defines a handler class built on `BaseHTTPRequestHandler`. There's no `vercel.json` in this repo and you don't need one for this setup.
3. Before the first deploy (or any time after, then redeploy), go to **Project Settings → Environment Variables** and add:
 - `ANTHROPIC_API_KEY` — your key from [console.anthropic.com](#) → API Keys. This is what `api/chat.py` reads server-side; the browser never sees it, same as `server.py` did locally with its `.env` file.
4. Deploy. Your app is live at the `*.vercel.app` URL Vercel gives you (or a custom domain if you attach one).

That's it — `bot.html`'s `fetch('/api/chat')` call doesn't know or care whether it's hitting `server.py` locally or `api/chat.py` on Vercel; both implement the same proxy contract.

Vercel's free (Hobby) tier covers this comfortably for personal use — 1M function invocations/month, more than enough for one person chatting with the assistant. It's meant for personal/non-commercial projects; if you ever want to put this in front of other people commercially, you'd need a paid plan. **Supabase's free tier** gives 500MB of Postgres storage and unlimited-enough auth for a personal calendar, but auto-pauses a project after a week of no activity — it unpauses itself the next time the app hits it, just with a few extra seconds of latency on that first request.

4. Local development (optional)

`server.py` still works for running the app on your own machine — `python3 server.py`, then open `http://localhost:5173`. It needs its own `.env` file in this folder with `ANTHROPIC_API_KEY=sk-ant-...` (never commit that file). Supabase itself needs no local setup either way, since `supabase-client.js` always talks to your real hosted Supabase project — there's no separate "local Supabase" step here.

`vercel dev` is the other option if you'd rather test against the exact same `api/chat.py` function Vercel will run in production, instead of `server.py`'s equivalent-but-separate implementation.

What's out of scope for now

- **Editing your name/avatar/password** on the Settings page — still shows "Coming soon" / "isn't available yet", same as before.

- **Syncing chat history, theme, and the voice-mute toggle across devices** — these deliberately stayed in `localStorage` (see `sql/schema.sql`'s bottom comment). Adding a `messages` table later, following the exact same `Cal.*`/RLS pattern already used for `events/tasks`, is a reasonably small follow-up if you want that.
- **Password reset / magic links / social login** — only `email+password` is wired up right now; Supabase Auth supports all of these, they just aren't built into `login.html` yet.

`PROJECT_STATE.md` in this folder documents the original `localStorage`-only architecture and is kept for historical reference — this file (`SETUP.md`) is the current source of truth for how the app actually works now.

Before you deploy: a couple of housekeeping files

`mock_test.js` and the `vendor/` folder in this zip were only used to verify this migration in a sandboxed test environment with no real network access — they're not part of the app and can be deleted before you push to GitHub/Vercel. Deleting them is optional (Vercel will just ignore them as static files), but they add unnecessary weight to the repo.